



RetroC Compiler

Compiler Architecture Document

Compiler Construction CS4031

Submitted to:

Mr. Zulfiqar Ali

Submitted by:

Amna Khan 23K-0856

Kissa Zehra 23K-0706

Afaq Ahmed 23K-0901

Section:

BCS-6B

1. Overview

The RetroC Compiler is a complete implementation of a compiler designed to translate high-level RetroC programs into an intermediate representation known as Three-Address Code (TAC), which is then executed on a custom-built virtual machine. RetroC is a simplified, C-inspired programming language developed to demonstrate the fundamental principles of compiler construction.

The compiler is implemented in C++ and follows a structured, multi-phase pipeline architecture. Each phase performs a specific transformation on the input program, gradually converting it from raw source code into executable form. This modular design improves clarity, maintainability, and extensibility of the system.

2. Overall Architecture

The RetroC compiler follows a sequential pipeline model in which the output of one phase serves as the input to the next. The overall flow begins with source code and progresses through lexical analysis, syntax analysis, semantic validation, intermediate code generation, optimization, and finally execution on a virtual machine.

This architecture ensures a clear separation of responsibilities among components and simplifies debugging and testing. Each module operates independently while contributing to the overall compilation process.

Phase	Source Files	Responsibility
Lexical Analysis	<code>scanner.cpp/.h</code>	Tokenizes raw source code into structured token objects
Syntax Analysis	<code>parser.cpp/.h</code>	Validates token sequences against the RetroC grammar
Semantic Analysis	<code>semantic.cpp/.h</code>	Enforces type rules, scope resolution, and logical correctness
IR Generation	<code>ir.cpp/.h</code>	Transforms validated AST into Three-Address Code (TAC)
Optimization	<code>optimizer.cpp/.h</code>	Applies constant folding and dead code elimination
Execution (VM)	<code>interpreter.cpp/.h</code>	Runs optimized TAC on a custom retro CPU simulator

3. Phase-wise Implementation

3.1 Lexical Analysis

Implemented in scanner.cpp and scanner.h

The lexical analysis phase is responsible for converting raw source code into a sequence of tokens. The scanner processes input character by character using a handcrafted state machine, identifying meaningful patterns such as keywords, identifiers, literals, operators, and punctuation symbols, and groups them into structured token objects. Whitespace and comments are discarded during this phase.

Key characteristics of the lexical analysis phase include:

- Handcrafted state machine for precise, efficient tokenization
- Maintenance of line and column information for accurate error reporting
- Immediate flagging of unrecognized characters as lexical errors

3.2 Syntax Analysis

Implemented in parser.cpp and parser.h

The syntax analysis phase validates whether the sequence of tokens conforms to the grammar of the RetroC language using a recursive descent parsing approach. The parser directly reflects the EBNF grammar and processes tokens through a hierarchy of functions corresponding to grammar rules, naturally enforcing operator precedence.

Notable design decisions include:

- Recursive descent approach that mirrors the EBNF grammar structure
- Syntax-directed translation enabling concurrent semantic analysis and IR generation
- Panic-mode error recovery that resumes parsing after statement delimiters

3.3 Semantic Analysis

Implemented in semantic.cpp and semantic.h

Semantic analysis ensures that the program is logically correct and adheres to the rules of the language. A central component of this phase is the symbol table, which maintains information

about variables, functions, and their respective scopes. The compiler supports block-level scoping, allowing nested scopes and variable shadowing.

Validation performed during this phase includes:

- Strict type checking to ensure operations are applied to compatible data types
- Declaration-before-use verification for all variables
- Function argument and return type validation
- Array indexing constraint checks where statically determinable

3.4 Intermediate Representation Generation

Implemented in `ir.cpp` and `ir.h`

The intermediate representation (IR) generation phase transforms the validated program into Three-Address Code (TAC). In TAC, each instruction contains at most three operands. Complex expressions are decomposed into smaller operations using temporary variables, making subsequent optimization and execution more manageable.

The IR generator also:

- Decomposes complex expressions into atomic three-operand instructions
- Introduces temporary variables to hold intermediate computation results
- Creates control-flow labels for conditional branching and loop constructs
- Produces a linear instruction sequence from structured high-level constructs

3.5 Optimization

Implemented in `optimizer.cpp` and `optimizer.h`

The optimization phase improves the efficiency of the generated TAC by eliminating redundant computations and unreachable code. Two primary optimization techniques are applied.

Optimization techniques applied:

- Constant Folding — evaluates expressions involving constant values at compile time, reducing runtime computation overhead
- Dead Code Elimination — removes instructions that can never be executed, such as statements following unconditional returns or jumps

3.6 Code Generation and Execution

Implemented in `interpreter.cpp` and `interpreter.h`

The final phase executes the optimized TAC using a custom virtual machine that simulates a simple retro CPU environment. The virtual machine executes instructions sequentially while maintaining an instruction pointer to track the current position. Control flow instructions modify this pointer to implement branching and looping.

The virtual machine provides the following capabilities:

- Sequential instruction execution with an instruction pointer for control flow
- Call stack management with activation records for each function invocation
- Support for recursion and nested function calls via the stack-based approach
- Storage of local variables, parameters, and return information per activation record

4. Command-Line Interface

The compiler provides a command-line interface implemented in `main.cpp`, allowing users to compile and execute RetroC programs efficiently. Available interface modes include:

- Direct execution — run a RetroC source file and display the output
- Output generation — produce files containing intermediate TAC and execution results
- Debug mode — trace execution and optimization steps for development and testing
- Interactive mode — test code snippets in real time via a REPL-style environment

5. Design Characteristics

The RetroC compiler is designed with modularity and simplicity as guiding principles. Each phase operates independently, making the system straightforward to understand and extend. The recursive descent parsing strategy simplifies grammar implementation, while the stack-based virtual machine provides a clear and transparent execution model.

The integration of semantic checks during parsing reduces overhead and ensures early error detection. The inclusion of basic optimization techniques further demonstrates practical improvements in compiled code efficiency.

6. Limitations

Despite its effectiveness as an educational tool, the compiler has certain intentional limitations. These constraints exist because the primary goal is to illustrate core compiler concepts rather than provide an industrial-strength system.

- Limited set of supported data types
- No advanced optimization techniques (e.g., register allocation, loop unrolling)
- Basic error recovery mechanisms only
- No memory management features such as garbage collection

7. Conclusion

The RetroC Compiler successfully demonstrates the complete compilation process, from source code analysis through to execution. By implementing all major phases within a unified system, it provides a comprehensive understanding of how programming languages are translated and executed.

Its structured design and simplified approach make it an effective platform for studying compiler construction and experimenting with language processing techniques. The modular pipeline ensures that each phase can be studied, tested, and extended independently, reinforcing sound software engineering principles alongside core compiler theory.